

AI-Assisted Workflow Optimisation for the HMM Transformer Research Project

C. L. (with Claude Code)

March 3, 2026

Contents

1	Introduction	1
2	Environment and Role of the AI Assistant	1
3	Python and PyTorch Conventions	2
3.1	Device Consistency	2
3.2	Minimal Complexity Principle	2
4	$\text{\LaTeX}$ Editing Conventions	2
5	Cluster, SLURM, and Sweep Submission	3
6	General Workflow Preferences	3
7	Automation Opportunities (Claude Max / Opus 4.6)	4
7.1	Whole-Codebase Context	4
7.2	End-to-End Experiment Automation	4
7.3	Report Drafting and Revision	4
7.4	Multi-Step Debugging Without Re-Explanation	4
7.5	Parallel Sub-Agents (Agentic Mode)	4
7.6	Persistent Memory and CLAUDE.md Maintenance	5
8	Summary and Next Steps	5

1 Introduction

This document records a set of workflow recommendations distilled from accumulated interactions between the researcher and Claude Code over the course of the HMM-transformer research project. It is intended as a living reference: a compact statement of conventions, guardrails, and automation opportunities that have been identified as useful for this particular combination of researcher background (physics), computing environment (Linux cluster accessed via SSH from macOS), and project structure (data generation $\rightarrow$ training $\rightarrow$ analysis pipeline).

The document is organised as follows. Section 2 describes the computing environment and the role of the AI assistant within it. Section 3 covers Python and PyTorch coding conventions enforced

by the assistant. Section 4 covers L^AT_EX editing conventions. Section 5 covers cluster and sweep submission practices. Section 6 states general workflow preferences. Section 7 discusses automation opportunities opened up by access to a higher-capability model tier (Claude Max / Opus 4.6).

2 Environment and Role of the AI Assistant

The researcher works on a Linux cluster via SSH from a macOS laptop. The following conventions apply.

- **Command suggestions** for remote (cluster) tasks are given as Linux/Bash terminal commands.
- **Keyboard shortcuts** for local (macOS) tasks use Mac notation (**Cmd**, **Opt**, etc.).
- The assistant **never assumes a local desktop GUI environment**; all file editing, job submission, and inspection happen in the terminal or via SSH.
- The project lives at `~/workspace/hidden-markov-chain/`; paths in scripts and configs should resolve from that root.

The AI assistant (Claude Code) acts as a collaborative coding partner with three roles in this project:

1. **Implementation** — writing and running code, generating config files, submitting cluster jobs.
2. **Explanation** — translating ML conventions and mathematical results into language accessible to a physicist.
3. **Documentation** — producing L^AT_EX reports, maintaining `CLAUDE.md` and `MEMORY.md` so that context persists across sessions.

3 Python and PyTorch Conventions

3.1 Device Consistency

When writing code that mixes PyTorch tensors from different sources (e.g. a tensor loaded from disk and one created in-flight during training), device placement (CPU vs. CUDA) must be made explicit.

- Use `.cpu()` before any NumPy comparison or `assert` that would fail silently if the tensor is on GPU.
- Use `.to(device)` consistently when passing tensors into model forward passes.
- Never rely on implicit broadcasting between a CPU and a CUDA tensor; PyTorch will raise a `RuntimeError`, but only at runtime, so the error may be hard to locate.

This convention was added explicitly because a physicist unfamiliar with the GPU memory model may not think to check device placement when a numerical result looks incorrect.

3.2 Minimal Complexity Principle

The assistant is instructed to prefer the simplest implementation that satisfies the current requirement. Concretely:

- Do not add error handling for scenarios that cannot occur given the current code structure.
- Do not introduce helper abstractions for one-off operations.
- Do not add docstrings or type annotations to code that was not modified in the current task.

This prevents unnecessary growth of the codebase and keeps PRs focused.

4 L^AT_EX Editing Conventions

When editing `.tex` files, the following checks are performed automatically by the assistant.

1. **Package dependencies:** any newly introduced symbol or environment must have its required package loaded in the preamble. Common cases that have arisen in this project:
 - `\gtrsim`, `\lesssim` → `amssymb`
 - `align`, `gather` environments → `amsmath`
 - `tikzpicture` → `tikz`
 - `tabularx`, `longtable` → corresponding packages
2. **Test compilation:** after non-trivial edits, attempt `pdflatex` compilation to catch package or syntax errors before the changes are finalised.
3. **Inline comments:** user comments of the form `\lcq{...}` are addressed before the document is considered complete. The comment markup is removed only upon explicit user approval.

The figure path convention in this project is `\graphicspath{{../../figures/<experiment>/}}`, placing figures in a top-level `figures/` directory organised by experiment name, separate from the `notes/` source tree.

5 Cluster, SLURM, and Sweep Submission

The cluster uses `addqueue` for job submission. When generating SLURM scripts or `wandb` sweep configs, the following checklist is applied.

1. **YAML numeric types:** values that look like numbers but should be treated as strings (e.g. experiment tags, run labels) must be quoted; otherwise the YAML parser silently converts them.
2. **Variable substitutions:** placeholders such as `${args}` in sweep agent launch commands must be present and syntactically correct.
3. **Environment loading:** module load commands and `conda activate / source .venv/bin/activate` lines must be included in every submission script, since cluster compute nodes do not inherit the login shell environment.
4. **File path resolution:** all paths in scripts (`save_dir`, `data_dir`, config paths) are verified to resolve correctly from the cluster filesystem root, not from a local working directory.

The main sweep tooling currently in use is `wandb sweep` with `bash submit_sweep.sh <sweep-id> [num_agents]`. All sweep configs live under `config/sweeps/`.

6 General Workflow Preferences

Implement, don’t only plan.

When asked to implement something, the assistant implements and runs it. A plan document is produced only if the user explicitly requests it. If a sweep or job submission is part of the task, it is submitted in the same session.

Ask for exact paths when searching fails.

If a first search attempt for a user-referenced file or function fails, the assistant asks the user for the exact file path rather than guessing at directory structures. This avoids silent errors from editing the wrong file.

Flag non-standard ML practices.

Because the researcher’s background is in physics, not machine learning, the assistant explicitly flags when a suggested approach deviates from standard ML practice, and briefly explains what is commonly done in the field.

Explain without coding by default.

When the user says “explain”, the assistant provides a conceptual explanation only. Code is written only if the user explicitly requests it in the same prompt.

7 Automation Opportunities (Claude Max / Opus 4.6)

Upgrading to Claude Max provides access to the Opus 4.6 model with a 200 000-token context window and higher rate limits. The following automation opportunities are directly enabled or substantially improved by this upgrade.

7.1 Whole-Codebase Context

With 200k tokens, the entire `src/` directory, all config files, and the active $\text{\LaTeX}$ report can be loaded simultaneously. This eliminates the need to manually identify which files are relevant to a given task, and allows the assistant to detect cross-file inconsistencies (e.g. a config constraint violation before a training run is submitted).

7.2 End-to-End Experiment Automation

A single assistant session can now cover a full experiment cycle:

1. Edit `base_config.yaml` for the new experiment.
2. Generate data via `mpirun data_generator.py`.
3. Submit training sweep via `wandb sweep` and `submit_sweep.sh`.
4. Monitor W&B run completion and pull results.
5. Run analysis scripts and produce figures.
6. Draft the $\text{\LaTeX}$ section summarising the results.

Steps 1–3 and 5–6 can be executed within one session without context loss.

7.3 Report Drafting and Revision

The assistant can hold the full report .tex file in context alongside the analysis outputs, allowing it to insert numerical results, update figures, and address \lq{} comments in a single pass. This is particularly useful for the iterative time_reversal/report.tex workflow where multiple rounds of edits are needed.

7.4 Multi-Step Debugging Without Re-Explanation

Longer context means the assistant retains the full stack trace, config, and relevant source code across multiple back-and-forth turns of debugging, without the user needing to re-paste context. This materially speeds up debugging cluster job failures and CUDA environment issues.

7.5 Parallel Sub-Agents (Agentic Mode)

The Max plan enables launching multiple sub-agents in parallel. Potential uses in this project:

- Run **forward** and **reversed** training sweeps concurrently and compare results.
- Simultaneously search the codebase for two independent issues.
- Generate figures and draft the corresponding L^AT_EX description in parallel.

7.6 Persistent Memory and CLAUDE.md Maintenance

With richer context, the assistant can more reliably detect when a new convention or experimental result should be recorded in `MEMORY.md` or `CLAUDE.md`, keeping the project knowledge base up to date without explicit prompting.

8 Summary and Next Steps

Table 1 provides a concise summary of the recommendations documented here.

Table 1: AI workflow recommendations for the HMM transformer project.

Category	Recommendation	Rationale
Environment	Use Linux commands for cluster tasks	SSH-only access
PyTorch	Explicit <code>.to(device)</code> / <code>.cpu()</code>	Silent GPU/CPU mismatches
L ^A T _E X	Check packages; compile after edits	Catch errors early
Cluster/Sweeps	Verify YAML types, paths, env loads	Cluster env differs from login shell
Workflow	Implement and run, not just plan	Reduces idle planning overhead
Workflow	Ask for exact path if search fails	Avoids editing wrong files
Max plan	Load whole codebase in context	Cross-file consistency checks
Max plan	Parallel sub-agents for sweeps	Forward/reverse in one session

Immediate next steps for adopting this workflow:

1. Add a **Makefile** or shell script that runs the full data $\rightarrow$ train $\rightarrow$ analyse pipeline with a single command, so the assistant can trigger it without constructing the individual commands each time.

2. Standardise figure generation scripts under `src/figures/` so the assistant knows where to look and what to run to regenerate any figure.
3. Consider a lightweight `results_registry.yaml` that records the W&B run ID, config hash, and key metrics for each completed experiment, serving as a machine-readable index the assistant can query.